University Management & Attendance System

A PROJECT REPORT

Submitted by

Khush Siddhpura[223SBECE30051]
Soni Arth[223SBECE30054]

In fulfilment for the award of the degree

Of

BACHELOR OF ENGINEERING

In

COMPUTER ENGINEERING

LDRP INSTITUTE OF TECHNOLOGY& RESEARCH, GANDHINAGAR

Kadi Sarva VishwaVidyalaya, Gandhinagar
2025-2026

LDRP Institute of Technology and Research

Computer Engineering Department

CERTIFICATE

This is to certify that the Project Work entitled **"University Management & Attendance System"** has been carried out by **Khush Siddhpura[223SBECE30051] Soni Arth [223SBECE30054]** under my guidance in fulfilment of the degree of Bachelor of Engineering in Computer Engineering (8th Semester) of Kadi Sarva VishwaVidyalaya University, Gandhinagar during the academic year 2025-26.

Guide:

Prof. Hetal Chavda
Internal Guide,
LDRP-ITR.

Prof. Ashish Patel
HOD,
LDRP-ITR.

Presentation - I for Project - III

1. Name & Signature of Internal Guide	
2. Comments from Panel Members	
3. Name & Signature of Panel Members	

Presentation - II for Project - III

1. Name & Signature of Internal Guide	
2. Comments from Panel Members	
3. Name & Signature of Panel Members	

Abstract:

Our main focus is to design a unique Student Management system that will improve Data management in Institutes, experience for both Students and the Administration authorities. The whole system will run on the web browser. The system is written in Python. Users will have the ability to log in from any place with internet connection. After that they will be able to access various resources and important notices, tasks provided by the faculties. Student Management System is a website which is helpful for students as well as the school/college authorities. In the current system all the activities are done manually. It is very time consuming and costly. Our Student Management System deals with the various activities related to the students.

List of figure

NO	NAME	PAGE NO
1	3.5 Class diagram	14
2	3.6 Use Case diagram	15
3	3.6.1 HOD Use case diagram	16
4	3.6.2 Staff Use case diagram	11
5	5.1 Database design diagram	19
6	5.2 ER Diagram	20
7	5.3 Data Flow Diagram	21
8	5.4.1 Attendance Mng Activity Diagram	22
9	5.4.2 Student registration Activity Diagram	22
10	5.5 Grade submission sequence diagram	23
11	9.1 HOD Home page	23
12	9.2 HOD Add staff page	24
13	9.3 HOD Add student page	24
14	9.4 Staff Home page	24
15	9.5 Staff take attendance	25
16	9.6 Staff leave application	25
17	9.7 Student home page	25
18	9.8 Student apply leave	26
19	9.9 Student feedback	26

TABLE OF CONTENTS

NO	CHAPTER NAME	PAGE NO
1	**Introduction**	5
	Introduction	6
	1.1Scope	7
	1.2.1Current Scope	8
	1.2.2 Future Scope	8
	1.2Project Summary and purpose	8
	1.3.1Project Summary	8
	1.3.2Purpose	8
	1.3Objectives	8
2	**Tools and Technology**	9
	2.1.1 Django Framework	9
	2.1.2 JavaScript:	9
	2.1.3 Oracle Database:	9
	2.2 Project Planning	10
	2.2.1 Project Development Approach:	10
	2.2.2 Milestones and Deliverables:	11
	2.1.3 Group Dependencies:	
3	**System Analysis**	12
	3.1 Study of current system	12
	3.2 Modules and Functionality of proposed system	12
	3.3 Feasibility Study	
	3.3.1 Technical feasibility	13
	3.3.2 Operational feasibility	13
	3.3.3 Economic feasibility	13
	3.4 Requirement Validation	13
	3.5 Class Diagram	14
	3.6 Use Case Diagrams	15
	3.6.1 Use Case Diagram of system	16
	3.6.1 Use Case Of HOD	16
	3.6.2 Use Case of Staff	17
		17
4	**Project Planning - Estimation and scheduling**	18
	4.1 Project Scheduling	
	4.2 Estimation	18
	4.2.1 Effort Estimation	18
	4.2.2 Cost Estimation	18
		18
5	**System Design**	19
	5.1 Database Design	19
	5.2 Entity Relationship Diagram	20
	5.3 Data Flow Diagram	21
	5.4 Activity Diagram	22
	5.5 Sequence Diagram	23
6	**Testing**	24
	6.1 Testing Plan	24

	6.1.1 Planning Steps	25
	6.2 Testing Strategies	26
	6.3 Test Cases	
7	**Experience, Limitations And Future Enhancement**	27
	7.1 Experience during project development	
	7.2 Future Enhancement	27
		27
8	**Bibliography**	28
	8.1 Conclusion	28
	8.2 Bibliography	28
9	**User Interface**	29

1. Introduction

- University material and Attendance is a website which is helpful for students as well as the school/college authorities. In the current system all activities are done manually. It is very time consuming and costly. Our system management deals with the various activities related to students.

- There are mainly 3 modules in this website
    - Faculty module
    - Student Module
    - Head of De

In the website we can register as a user and the user has two types, student and administrator. Administrator has the power to add, delete new users or manipulate the controls of a student user account. A student can register as a user and can access various resources such as study materials, notices, time-table, upload his/her work, edit and delete his profile. The administrator can inform all students about any particular event or study related affair by uploading a notice and also provide them a time-table for their regular study schedule or any examination. All the users can see the attendance.

1.1 System Analysis

EXISTING SYSTEM:

System Analysis is a detailed study of the various operations performed by a system and their relationships within and outside of the system. Here the key question is- what all problems exist in the present system? What must be done to solve the problem? Analysis begins when a user or manager begins a study of the program using an existing system.

During analysis, data collected on the various files, decision points and transactions handled by the present system. The commonly used tools in the system are Data Flow Diagram, interviews, etc. Training, experience and common sense are required for collection of relevant information needed to develop the system. A good analysis model should provide not only the mechanisms of problem understanding but also the framework of the solution. Thus, it should be studied thoroughly by collecting data about the system. Then the proposed system should be analysed thoroughly in accordance with the needs.

System analysis can be categorised into four parts.

- System planning and initial investigation
- Information Gathering
- Applying analysis tools for structured analysis
- Feasibility study
- Cost/ Benefit analysis.

In the current system we need to keep a number of records related to the student and want to enter the details of the student and the attendance manually. In this system only, the teacher or the school authority views.

1.2 PROPOSED SYSTEM:

In our proposed system we have the provision for adding the details of the students by themselves. So, the overhead of the school authorities and the teachers has become less. Another advantage of the system is that it is very easy to edit the details of the student and delete a student when it is found unnecessary. The marks of the student are added in the database and so students can also view the marks whenever they want. Our proposed system has several advantages.

- User friendly interface
- Fast access to database
- Less error
- More Storage Capacity

Look and Feel Environment Quick Transaction All the manual difficulties in managing the student details in a school or college have been rectified by implementing computerisation.

1.3 FEASIBILITY ANALYSIS

Whatever we think need not be feasible. It is wise to think about the feasibility of any problem we undertake. Feasibility is the study of impact, which happens in the organisation by the development of a system. The impact can be either positive or negative. When the positives nominate the negatives, then the system is considered feasible. Here the feasibility study can be performed in two ways such as technical feasibility and Economical Feasibility.

TECHNICAL FEASIBILITY:

We can strongly say that it is technically feasible, since there will not be much difficulty in getting required resources for the development and maintaining the system as well. All the resources needed for the development of the website as well as the maintenance of the same are available in the organisation where we are utilising the resources which are available already.

ECONOMIC FEASIBILITY:

Development of this website is highly economically feasible. The organisation needed not spend much money for the development of the system already available. The only thing to be done is making an environment for development with effective supervision. If we are doing so, we can attain the maximum usability of the corresponding resources. Even after the development, the organisation will not be in condition to invest more in the organisation. Therefore, the system is economically feasible

2 Tools and Technology

2.1.1 Django Framework:

- Django is a high-level Python web framework that enables rapid development of secure and maintainable websites.
- It encourages clean and pragmatic design, and follows the "don't repeat yourself" (DRY) principle.
- It provides a ready-to-use admin interface, authentication support, and routing capabilities, allowing developers to focus on their application's business logic.
- With Django, developers can build dynamic web applications, handling backend processes efficiently, including database management with its built-in ORM.
- It integrates with popular databases like PostgreSQL, MySQL, SQLite, and Oracle.

2.1.2 JavaScript:

- JavaScript, along with AJAX, is used to create asynchronous web applications, allowing web pages to update content without reloading the entire page.
- AJAX allows small amounts of data to be exchanged between the client and server in the background, improving user experience and application performance.
- This is essential for applications that require real-time interaction, such as updating student attendance without refreshing the page.

2.1.3 Oracle Database:

- Oracle is an object-relational database management system used to store and retrieve large sets of data efficiently.
- It supports the SQL programming language and provides robust data storage and transaction management, which is critical for handling student information, attendance records, and course data.
- Oracle's scalability and reliability make it ideal for large-scale systems such as a university management system where multiple operations are performed simultaneously.
- It supports PL/SQL, enabling developers to create stored procedures to optimize database performance.

2.2 Project Planning

2.2.1 Project Development Approach:

- The **INCREMENTAL model** is chosen for project development.
- It combines elements of the waterfall model and allows iterative development where the system is built and delivered in pieces or increments.

- This ensures that feedback from each phase can be integrated into future stages, improving flexibility and allowing for changes in requirements.
- The phases include:
    1. **Communication**: Gathering and understanding system requirements.
    2. **Planning**: Estimating effort, timelines, and resources.
    3. **Modeling**: System design and architecture development.
    4. **Construction**: Coding, system integration, and testing.
    5. **Deployment**: Delivering increments and gathering user feedback.

2.2.2 Milestones and Deliverables:

Month 1:

- **Milestones:**
    - Requirement analysis and project planning.
    - Define the core objectives and goals.
- **Deliverables:**
    - Analysis report of user needs and system requirements.

Months 2-4:

- **Milestones:**
    - System analysis and design.
    - Creation of essential diagrams such as Data Flow Diagrams (DFD), Entity-Relationship Diagrams (ERD), etc.
- **Deliverables:**
    - System Requirements Specification (SRS).

Months 5-7:

- **Milestones:**
    - Develop the HOD, Staff, and Student modules.
    - Database setup with Oracle, ensuring smooth integration.
- **Deliverables:**
    - Database creation, coding for user authentication, and role-based access control.

Month 8:

- **Milestones:**
    - Complete integration of the modules.
    - Testing the system for bugs and functionality issues.
- **Deliverables:**
    - Tested system ready for deployment, user feedback integration, final code review.

2.1.3 Group Dependencies:

- The team follows a **democratic decentralized structure**.
- Decisions are made collaboratively without a single leader; members contribute based on expertise and availability.
- Regular consultations with project guides and external advisors ensure that any critical decision is well-informed and aligned with the project goals.

3. System Analysis for University Management & Attendance System

3.1 Study of Current System:

- The current university management system relies heavily on manual processes, spreadsheets, and paperwork for managing student records, attendance, and communication between staff and students.
- **Challenges in Current System**:
  - Lack of centralized access to student data.
  - Difficulty in tracking attendance and managing student performance.
  - Inefficient leave management system.
  - HOD and Staff overwhelmed with manual tasks.
  - Delays in approvals and administrative processes.
- **Objective**: To create an automated, centralized system to streamline student management processes, reduce manual work, and enhance data accuracy.

3.2 Modules and Functionality of Proposed System:

1. **HOD Module**:
   - Add, edit, and manage staff and student records.
   - View overall attendance reports.
   - Approve or reject student leave requests.
   - Manage courses and assign subjects.
2. **Staff Module**:
   - Manage assigned students' records and update attendance.
   - Add student results and assessments.
   - View leave requests and take necessary actions.
3. **Student Module**:
   - View personal attendance reports.
   - Apply for leaves and track application status.
   - Access results and other academic-related information.

3.3 Feasibility Study:

3.3.1 Technical Feasibility:

- The project leverages technologies like Django for backend development and Oracle for database management, ensuring scalability and security.
- With the **incremental development model**, each system module will be implemented and tested iteratively, reducing risks during deployment.

- Existing technology stacks (like Django, AJAX, Oracle) are well-supported, ensuring smooth integration and development.

3.3.2 Operational Feasibility:

- The system will significantly reduce the workload of staff and HOD by automating most administrative processes.
- User-friendly interfaces ensure that students, staff, and HOD can easily navigate and use the system.
- As the system addresses the pain points of the current manual processes, user acceptance is expected to be high.

3.3.3 Economic Feasibility:

- **Initial Costs**: Development, testing, and deployment of the system will require investment in software licenses, cloud hosting, and development resources.
- **Long-term Benefits**: The system will reduce administrative costs, time spent on manual tasks, and errors, leading to long-term savings.
- **Return on Investment**: The system offers a high ROI by improving operational efficiency, reducing delays, and optimizing resource allocation.

3.4 Requirement Validation:

- Stakeholders (students, staff, HOD) have validated the system requirements.
- A prototype was reviewed by the HOD, staff members, and students to ensure the proposed solution meets the needs of each user group.
- Any additional features or corrections identified during this process will be addressed during iterative cycles.

3.5 Class Diagram:

- The class diagram represents the structure of the system, including the relationships between entities such as **Student**, **Staff**, **HOD**, **Course**, and **Attendance**.
- Key entities include:
    - **Student**: Attributes include student ID, name, attendance, results, etc.
    - **Staff**: Attributes include staff ID, name, and subjects handled.
    - **HOD**: Attributes include HOD ID, name, department.
    - **Course**: Includes course ID, course name, and associated subjects.

- **Attendance**: Tracks student attendance data for each class and subject

3.6 Use Case Diagrams:

3.6.1 Use Case Diagram of the System:

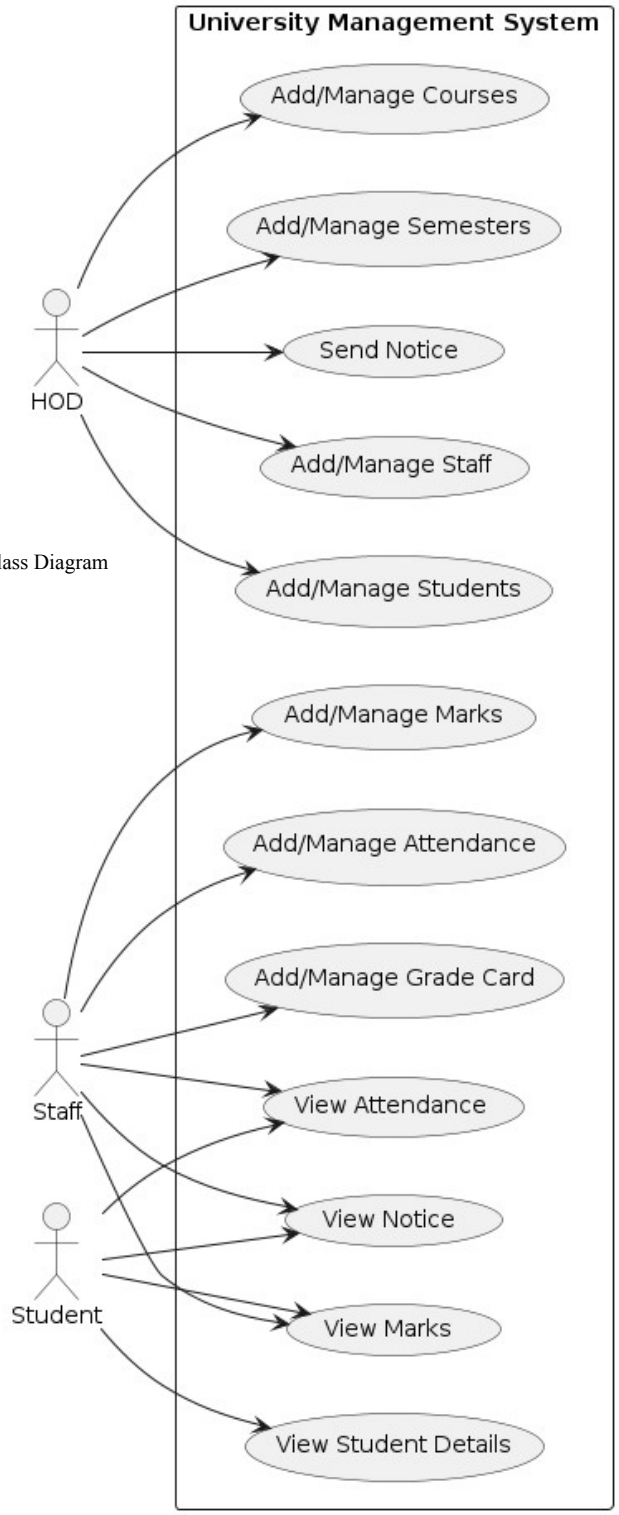
- The overall use case diagram includes the main actors: **HOD**, **Staff**, and **Students**, and shows interactions with the system for each role.

- **HOD Use Cases**: Manage staff, view attendance, approve leave requests, manage courses.

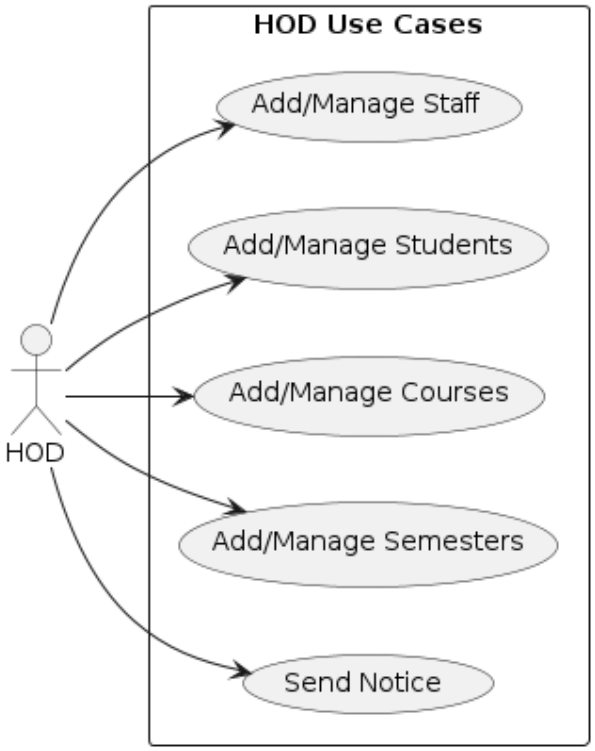
- **Staff Use Cases**: Manage students, update attendance, add results, view leave requests.

- **Student Use Cases**: View attendance, apply for leave, view results.

3.5 Class Diagram



3.6. Use case diagram



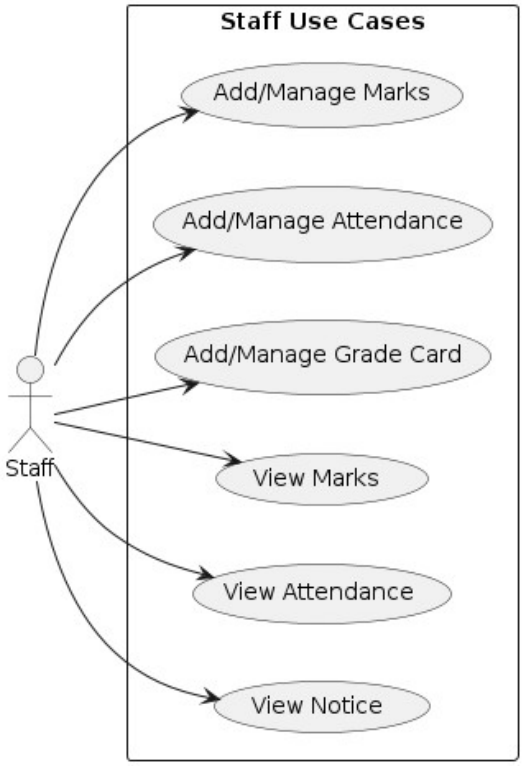
3.6.1 HOD Use case diagram

3.6.1 Use Case Scenarios and Diagram of HOD Module:

- **Scenario**: The HOD logs into the system and views the overall student attendance reports for all subjects. If a student's attendance is below the required threshold, the HOD informs the concerned staff.
- **Use Case Diagram**: Shows HOD interacting with the system to view
- attendance, approve leave, and manage staff.

3.6.2 Use Case Scenarios and Diagram of Staff Module:

- **Scenario**: A staff member logs into the system and updates the attendance for their assigned class. They then review a student's leave request and mark it as approved or rejected based on the student's attendance record.
- **Use Case Diagram**: Illustrates staff interactions with managing attendance, adding results, and processing leave requests.



3.6.2 Staff Use Case diagram

4 Project Planning - Estimation and Scheduling

4.1 Project Scheduling:

- **Project scheduling** ensures timely completion of each phase within set timeframes, following the **Incremental Model**.
- **Phases**:
  1. **Requirement Gathering and Analysis**: 2-4 weeks.
  2. **System Design**: 4-6 weeks (including diagrams and database design).
  3. **Module Development** (HOD, Staff, Student): 4-6 weeks per module.
  4. **Testing and Debugging**: 2-3 weeks per module.
  5. **Final Integration and Testing**: 3-4 weeks.
  6. **Deployment and Maintenance**: Final 2 weeks.
- **Milestones**:
  - **Month 1**: Completion of requirement analysis.
  - **Months 2-3**: System design and prototyping.
  - **Months 4-6**: Module development.
  - **Month 7**: System integration and testing.
  - **Month 8**: Deployment, user training, and documentation.

4.2 Estimation:

4.2.1 Effort Estimation:

- **Effort Estimation** uses the **COCOMO Model** to calculate the number of person-hours required to complete the project based on code size and complexity.
- For UFT, each module (HOD, Staff, Student) will have its effort calculated individually.
- **Effort Estimation Formula (COCOMO Basic Model)**:
  - **Effort (person-months)** = 2.94 * (KLOC)^E, where:
    - **KLOC**: Estimated lines of code (thousands).
    - **E**: Complexity exponent based on project size.
- Example: If the project has 30,000 lines of code (KLOC = 30), the estimated effort will be calculated based on complexity and other factors.

4.2.2 Cost Estimation:

- **Cost Estimation** includes development, testing, and maintenance costs. The estimation will be calculated in Indian Rupees (₹).
- **Factors Considered**:
  1. **Development Costs**:
     - Developer salaries.
     - Software licenses (e.g., Django, Vs-Code).
     - Cloud/server costs for hosting.
  2. **Testing Costs**:

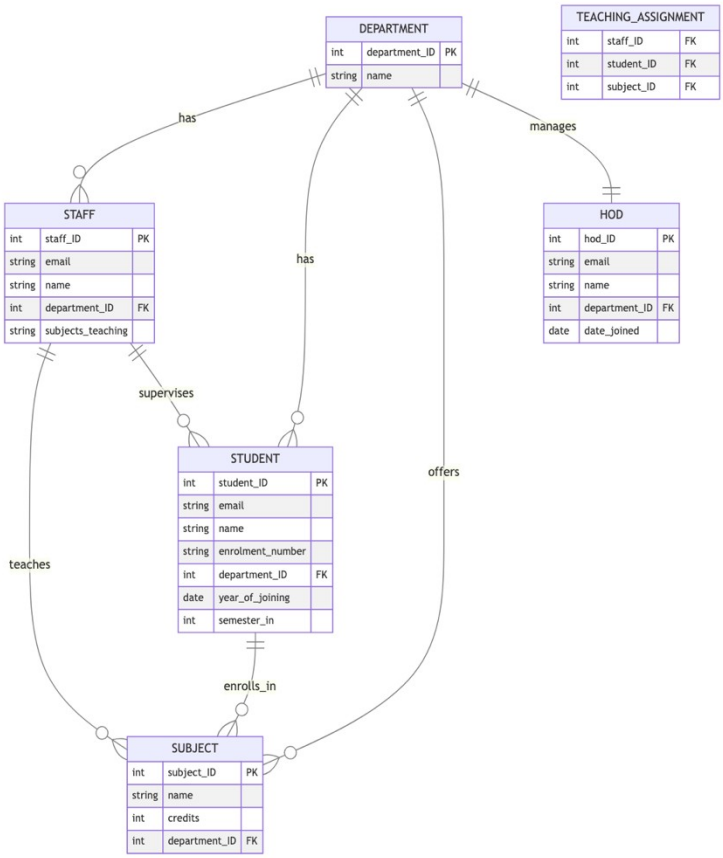
- Tools for testing and resources.
- Bug fixes during and after development.

3. **Maintenance Costs**:
   - Ongoing support and infrastructure management.
   - Server/database maintenance.

5. System Design

5.1 Database Design

- **Database Design** defines the structure of data storage in the **University Management & Attendance System** system.
- Key Entities and Tables:
  - **Users Table**: Stores details of HODs, staff, and students.
  - **Department Table**: Contains information about all the department offered by the university.
  - **Subjects Table**: Lists all the subjects associated with each course.
  - **HOD Table**: Adds and manages staff and students and sends notice.
  - **Staff Table:** Adds attendance and marks for students.
  - **Staff Table:** Can view their attendance and marks.



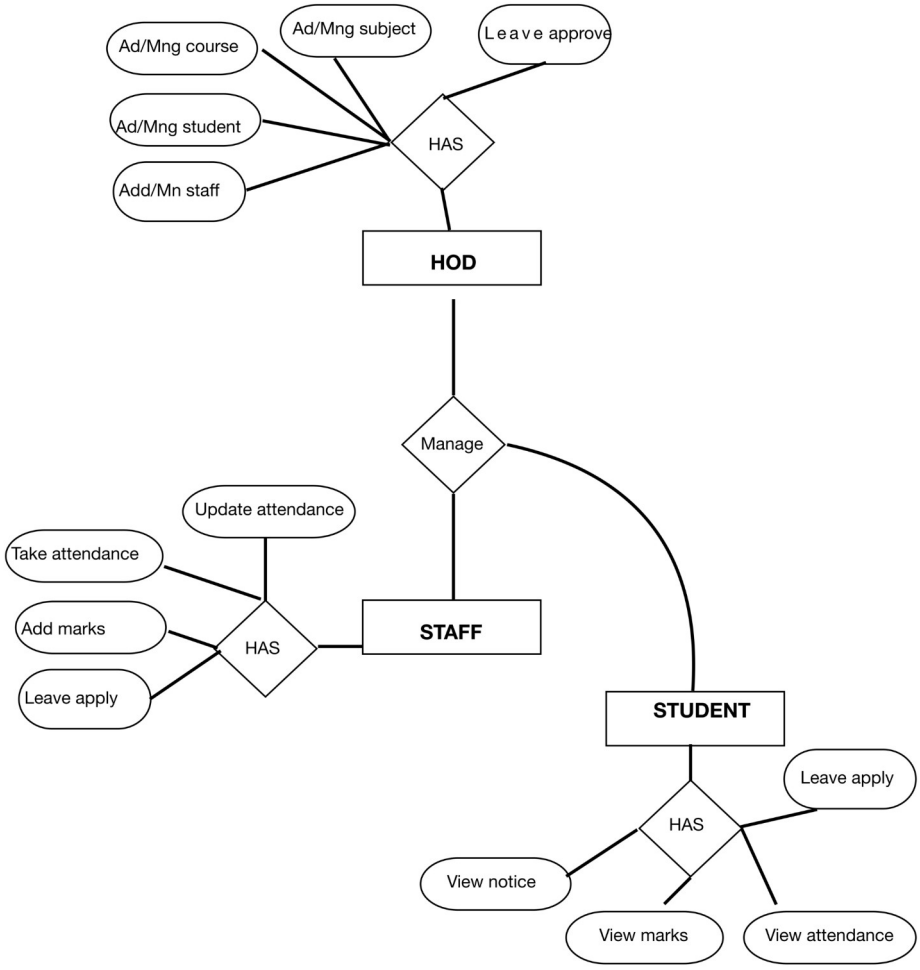
5.1 Database design diagram

- Each entity has a unique **primary key** (like User ID, Course ID) and **foreign keys** to establish relationships between tables (e.g., a course may have multiple subjects).

5.2 Entity Relationship Diagram (ERD)

- **ER Diagram** shows the relationships between different entities in the system:
  - **Entities**: Users (HOD, Staff, Students), Courses, Subjects, Attendance, Results, Leave Applications.
  - **Relationships**:
    - A **HOD** manages multiple **Staff** members.
    - **Staff** teaches multiple **Subjects** and **Courses**.
    - **Students** are enrolled in **Courses** and have **Attendance** records for each **Subject**.

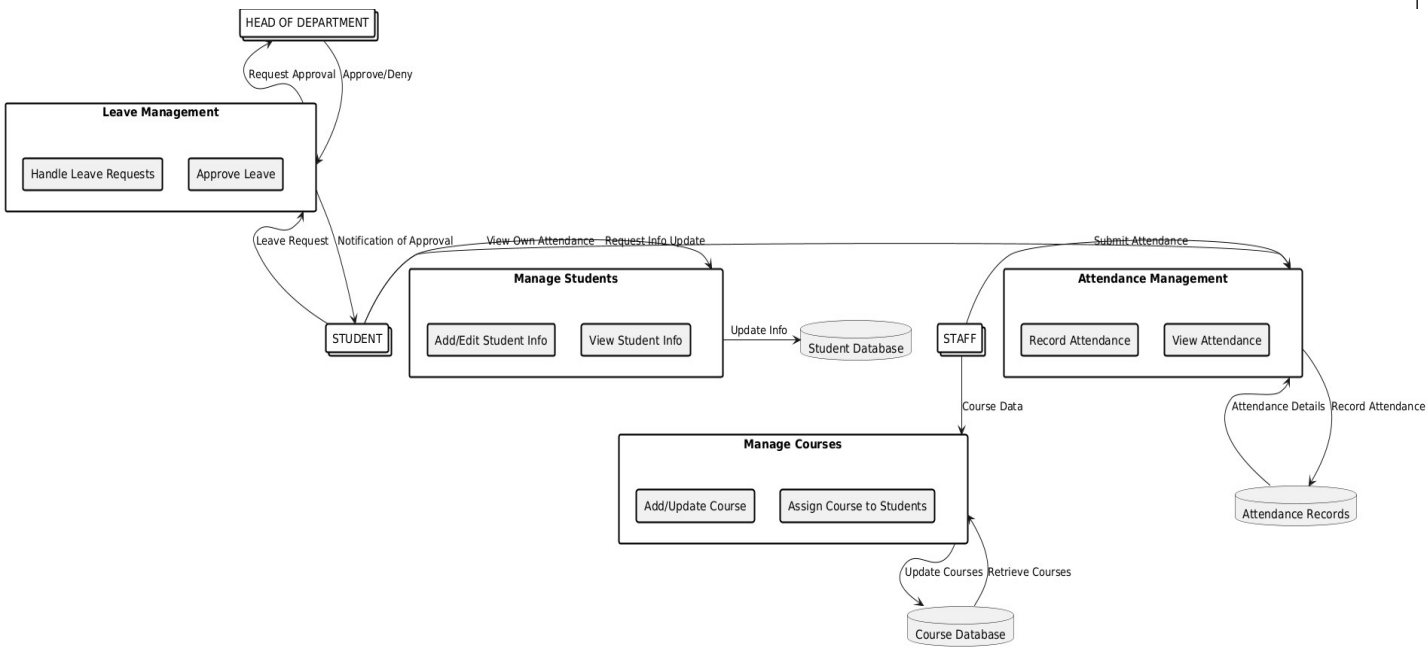
- **Results** link **Students**, **Courses**, and **Subjects**.
  - The **ERD** visually represents entities as rectangles and relationships as lines connecting them, with notations for one-to-many or many-to-many relationships.



5.2 ER Diagram

5.3 Data Flow Diagram (DFD)

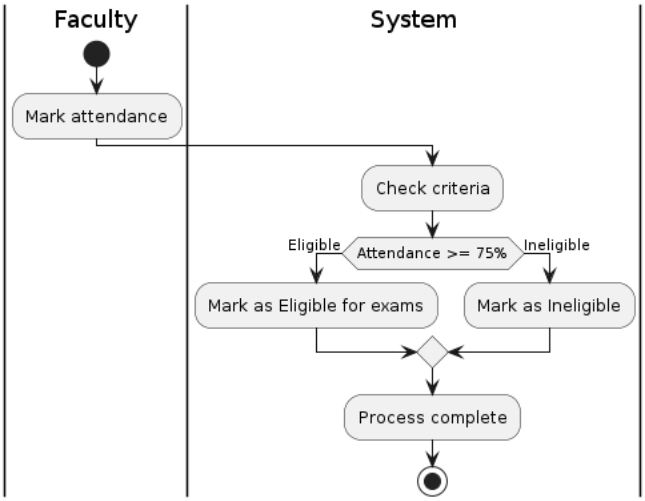
- The **DFD** represents the flow of data within the system across different modules.
  - **Level 0 (Context Diagram)**: High-level view showing interactions between external users (HOD, Staff, Students) and the system.
  - **Level 1 (Decomposition)**:
    - **HOD Module**: Manages staff, views reports, and monitors student progress.
    - **Staff Module**: Inputs attendance and results, manages students.
    - **Student Module**: Applies for leave, views attendance and results.



5.3 Data Flow Diagram

5.4 Activity Diagram

- The **Activity Diagram** represents the workflow of key activities in the system:
  - **HOD Activities**: Approving leave, managing staff.
  - **Staff Activities**: Taking attendance, entering results.
  - **Student Activities**: Applying for leave, viewing attendance and results.



5.4.1 Attendance Mng Activity Diagram

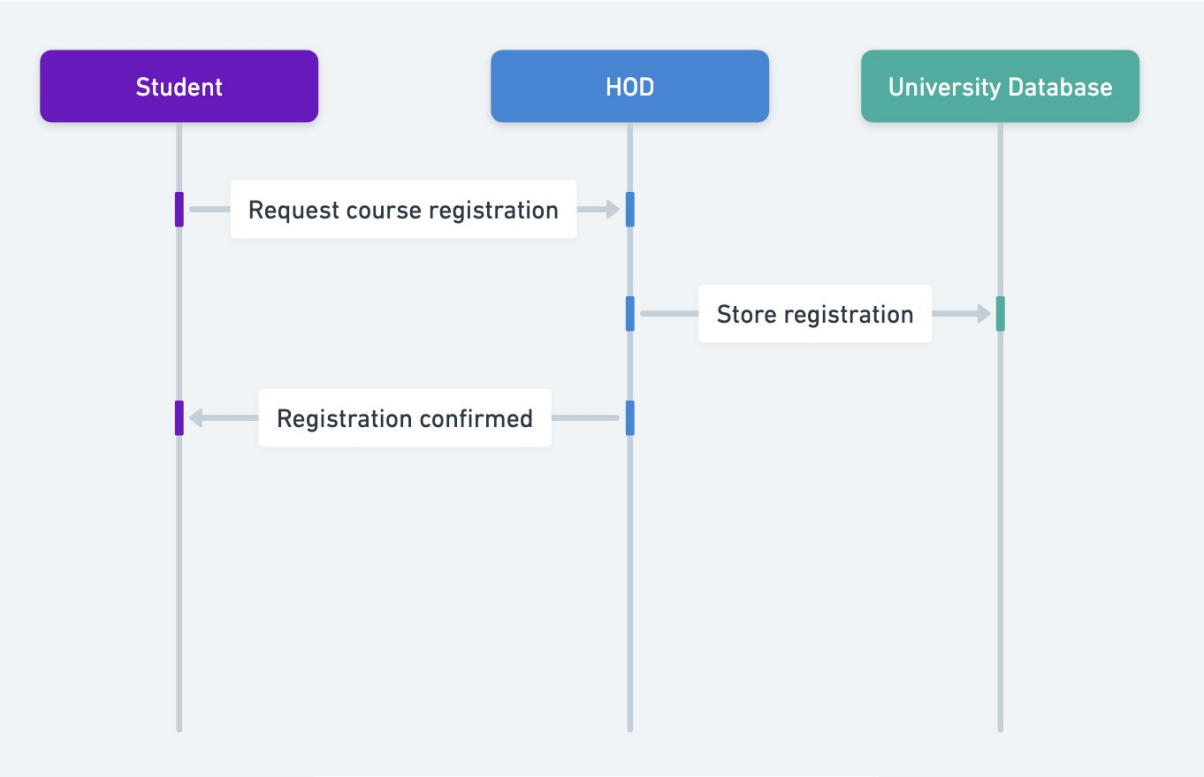
- **Example**:
  - **Leave Application Process**:
    1. Student submits a leave request.
    2. Request is sent to the staff member for review.
    3. The staff forwards the request to the HOD for approval.

5.5 Sequence Diagram for Grade Submission

The sequence diagram for the Grade Submission process in the University Management & Attendance System system will outline the step-by-step interactions between different users (Staff, Student) and the system.

Example: Grade Submission Process

1. Staff logs into the system.
2. Staff selects the course and subject for which they want to submit grades.
3. The System retrieves the list of enrolled students for the selected course and subject.
4. Staff enters the grades for each student.
5. The System validates the input data (e.g., correct grade format, completeness).
6. Once validated, the System updates the Results Table with the submitted grades.
7. System notifies the students by updating their grade reports in the Student Module.
8. Student can view their grades by logging into the system.



5.5 Grade submission sequence diagram

5.6 Data Dictionary

5.6.1 HOD data table

Attribute	Size	Constrain
Id	Number(10)	Primary key
Name	Varchar(10)	
Department	Varchar(10)	
Joined	Varchar(10)	

5.6.1 HOD data table

5.6.2 Staff data table

Attribute	Size	Constrain
Id	Number(10)	Primary key
Name	Varchar(10)	
Department	Varchar(10)	
Joined	Varchar(10)	
Course	Varchar(10)	
Subject_under	Varchar(10)	

5.6.2 Staff data table

5.6.3 Student data table

Attribute	Size	Constrain
Id	Number(10)	Primary key
Name	Varchar(10)	
Department	Varchar(10)	
Joined	Varchar(10)	
Course	Varchar(10)	
Mentor	Varchar(10)	

5.6.3 Student data table

5.6.4 Subject data table

Attribute	Size	Constrain
Id	Number(10)	Primary key
Name	Varchar(10)	
Department	Varchar(10)	
Staff_Under	Varchar(10)	

5.6.4 Subject data table

5.6.5 Department data table

Attribute	Size	Constrain
Id	Number(10)	Primary key
Name	Varchar(10)	
HOD	Varchar(10)	

5.6.5 Department data table

5.6.6 Course data table

Attribute	Size	Constrain
Id	Number(10)	Primary key
Name	Varchar(10)	
Department	Varchar(10)	

5.6.6 Course data table

6.Testing

-

6.1 Testing Plan

- The **Testing Plan** outlines the approach to ensure the **University Management & Attendance System** system is reliable, functional, and meets user requirements.

6.1.1 Planning Steps

1. **Define Objectives**:
   - Ensure all system functionalities work as intended.
   - Validate data accuracy and integrity, particularly in key modules (HOD, Staff, Student).
   - Assess performance under different user loads.
2. **Identify Testing Types**:
   - **Unit Testing**: Testing individual components or modules for expected functionality.
   - **Integration Testing**: Ensuring different modules (HOD, Staff, Student) work together seamlessly.
   - **System Testing**: Validating the complete system's compliance with the specified requirements.
   - **User Acceptance Testing (UAT)**: Engaging end-users (HODs, Staff, Students) to validate usability and functionality.
3. **Develop Testing Schedule**:
   - Set timelines for each phase of testing, allowing for re-testing and bug fixes.
4. **Assign Roles and Responsibilities**:
   - Allocate tasks to team members, designating who will execute and monitor each testing phase.
5. **Prepare Testing Environment**:
   - Set up the necessary infrastructure, including servers, databases, and test data.

6.2 Testing Strategies

- **Testing strategies** involve the methodologies and techniques used to ensure comprehensive coverage.

1. **Manual Testing**:
   - Conduct manual tests for user interfaces and complex scenarios that require human judgment.
2. **Automated Testing**:
   - Implement automated testing tools (e.g., Selenium, TestNG) for repetitive tasks such as regression testing.
3. **Performance Testing**:
   - Use tools (e.g., JMeter) to simulate multiple users and assess the system’s performance under load.
4. **Security Testing**:

- Conduct tests to identify vulnerabilities (e.g., SQL injection, XSS) and ensure data protection.

5. **Regression Testing**:
   - Retest the system after changes or bug fixes to confirm that existing functionalities remain unaffected.

6.3 Test Cases

- **Test Cases** are specific conditions or variables under which a tester will determine if a system meets requirements.

Example Test Cases :

Test Case ID	Description	Input	Expected Output	Status
TC-01	Validate Staff login functionality	Valid credentials	Access granted to Staff Dashboard	Pass
TC-02	Verify grade submission by Staff	Grade entries for students	Grades saved successfully in the Results Table	Pass
TC-03	Check attendance marking functionality	Attendance data	Attendance recorded for students	Pass
TC-04	Test leave application by Student	Leave request details	Request submitted successfully	Pass
TC-05	Confirm that students can view their grades	Student login	Display of accurate grade reports	Pass
TC-06	Validate HOD's ability to approve leave	Leave request	Status updated to approved/rejected	Pass
TC-07	Performance under multiple simultaneous logins	Multiple concurrent users	System remains responsive and functional	Pass
TC-08	Security check against SQL injection	Malicious SQL input	No database access, error message displayed	Pass

6.1 Test case example

7.Experience, Limitations, and Future Enhancement

7.1 Experience During Project Development

- **Collaboration**:
  - Worked closely with team members and faculty to gather requirements, which fostered effective communication and a deeper understanding of user needs.
  - Engaged in regular meetings to discuss progress, challenges, and feedback, creating a collaborative environment.
- **Skill Development**:
  - Gained hands-on experience with various technologies, including Django framework for backend development, Oracle Database for data management, and AJAX for enhancing user experience.
  - Improved problem-solving skills by addressing challenges during integration and testing phases.
- **Project Management**:
  - Implemented the Incremental Model, allowing for iterative development and timely feedback from stakeholders.
  - Developed essential project management skills, including scheduling, resource allocation, and milestone tracking.
- **Learning Curve**:
  - Encountered challenges in understanding complex database interactions and optimizing performance but overcame them through research and collaboration.
  - Gained valuable insights into user experience design, ensuring the application is intuitive and accessible.

7.2 Limitations

- **Resource Constraints**:
  - Limited availability of resources and tools affected the depth of testing and performance optimization.
  - Restricted access to hardware for testing real-world scenarios impacted the ability to simulate high-load conditions effectively.
- **Scope Creep**:
  - New feature requests during development led to adjustments in timelines and scope, affecting project delivery.
  - Balancing essential features with additional functionalities proved challenging.
- **User Adoption**:
  - Some end-users (faculty and students) had difficulty adapting to the new system due to varying levels of technical proficiency.
  - The need for extensive training sessions was identified to enhance user comfort and acceptance of the new system.
- **Documentation**:
  - Inadequate documentation during the development phase posed challenges in maintenance and knowledge transfer among team members.

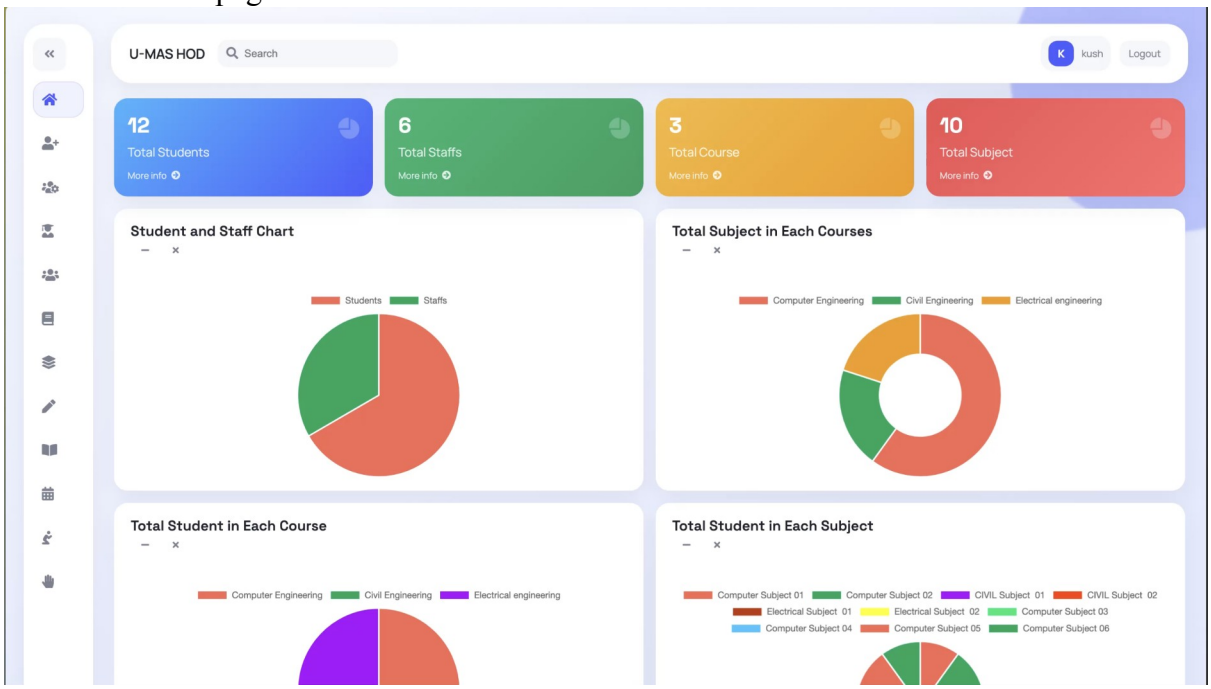
- Initial requirements gathering did not capture all user needs, leading to rework in later stages.

7.3 Future Enhancement

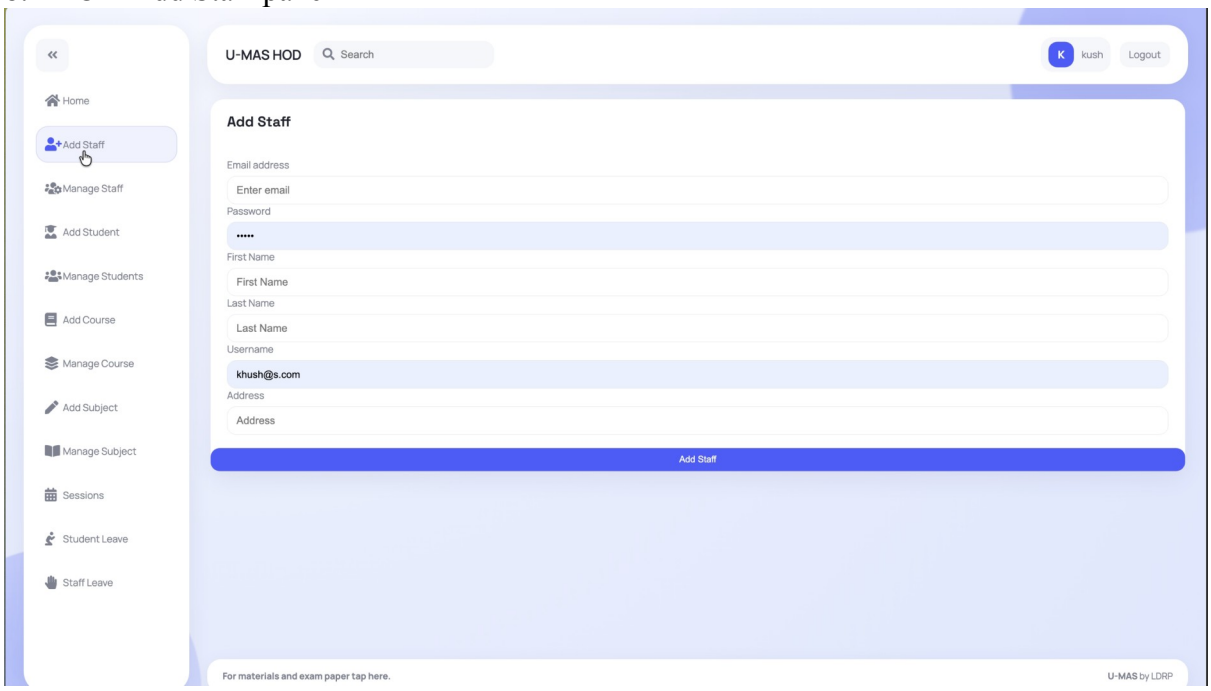
- **User Training and Support**:
  - Develop comprehensive training modules and resources to assist users in adapting to the system effectively.
  - Implement a support system for real-time assistance and troubleshooting.
- **Feature Expansion**:
  - Integrate additional functionalities, such as a mobile application version of UFT, to increase accessibility and user engagement.
  - Consider implementing advanced reporting and analytics features to provide insights into student performance and administrative efficiency.
- **Enhanced User Experience**:
  - Focus on improving the UI/UX based on user feedback, ensuring a more intuitive and responsive design.
  - Conduct user surveys to gather input for continuous improvement and feature prioritization.
- **Performance Optimization**:
  - Investigate and implement performance tuning strategies to handle increased user load effectively.
  - Optimize database queries and application architecture for better efficiency and speed.
- **Regular Updates and Maintenance**:
  - Establish a regular maintenance schedule to address bugs, implement updates, and keep the system secure.
  - Plan periodic reviews of the system to assess functionality against evolving user needs and technological advancements.

8. User Interface

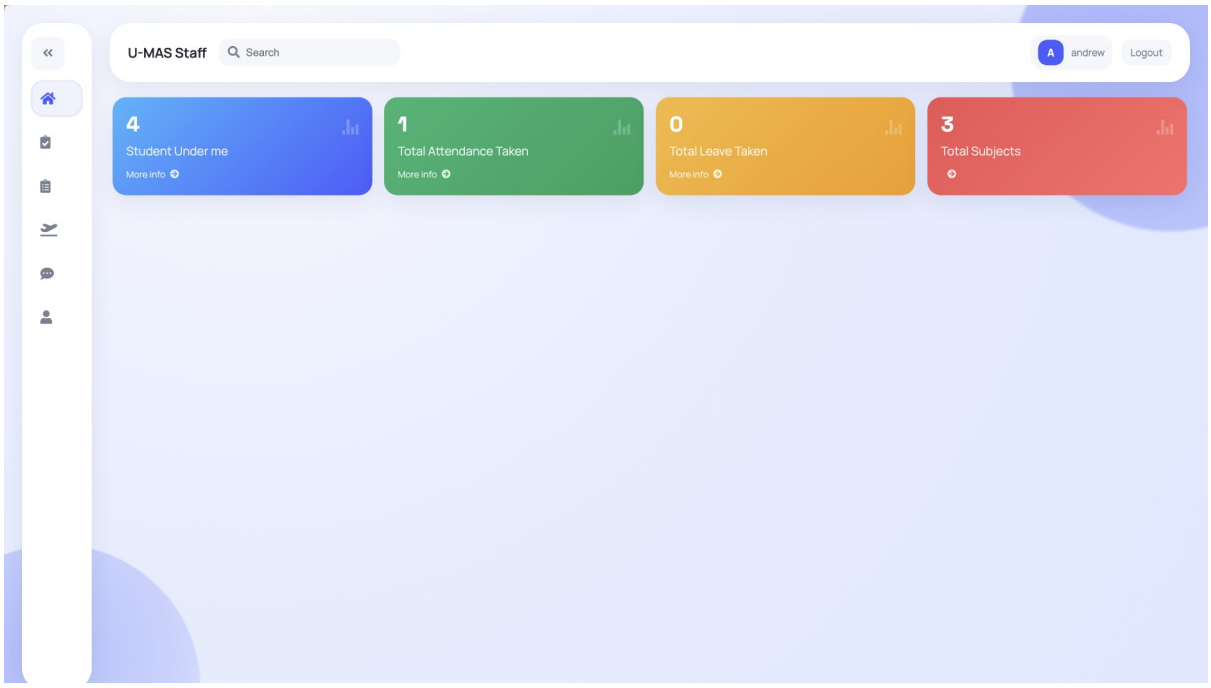
8.1 HOD Home page Interface



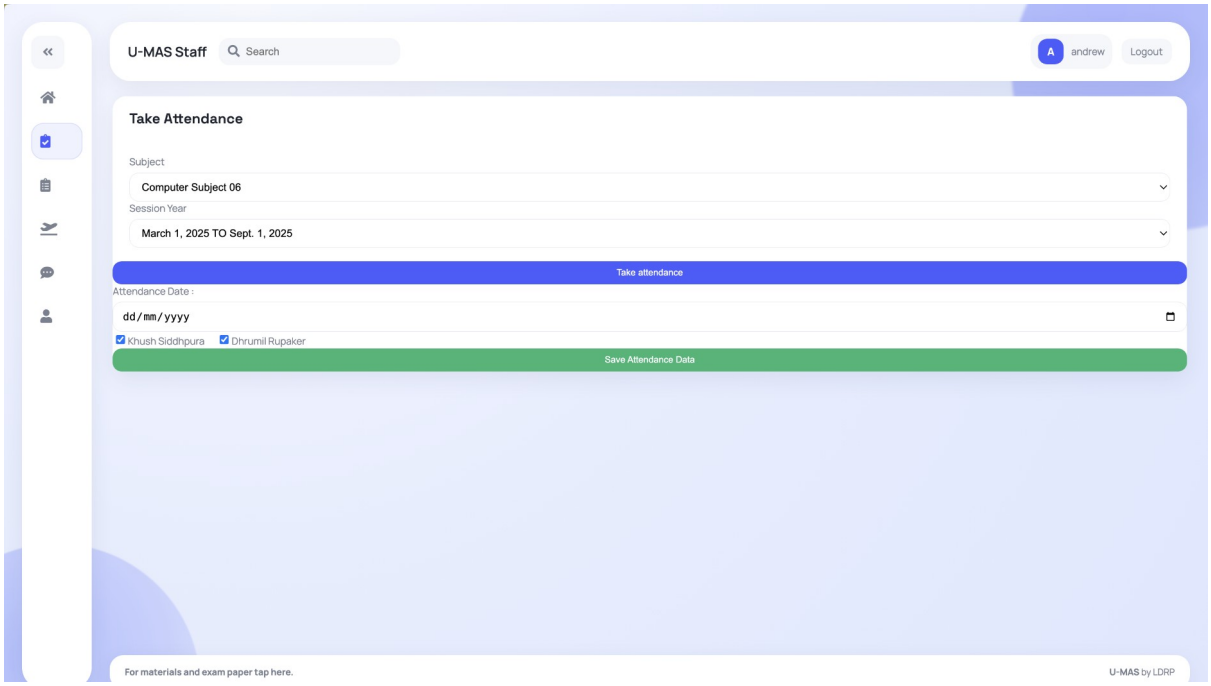
8.2 HOD Add Staff panel



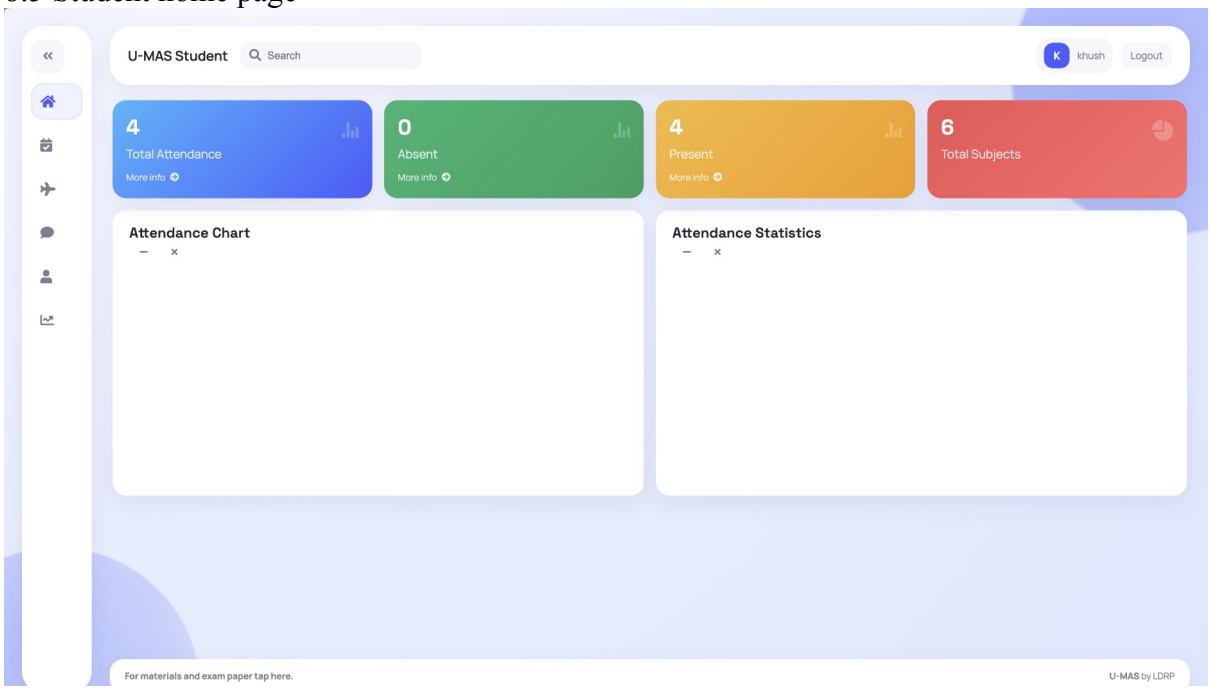
8.3 Staff Home Page



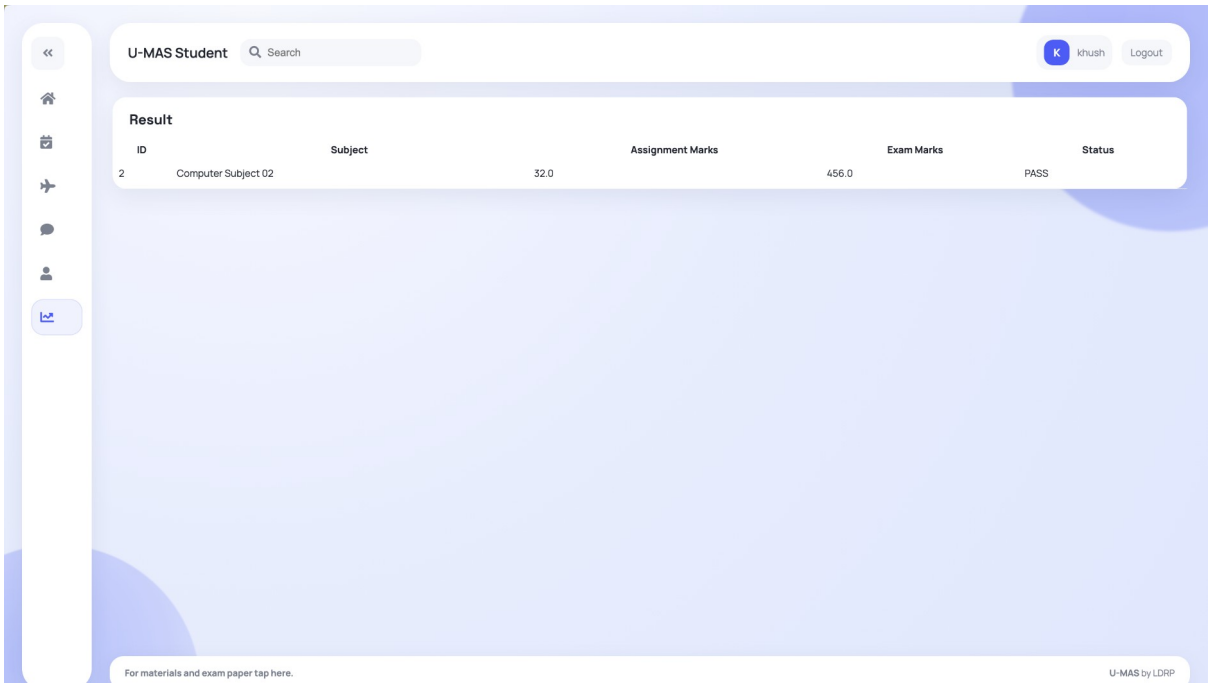
8.4 Staff Take Attendance panel



8.5 Student home page



8.6 Student Result Panel



9.Bibliography

9.1 Conclusion

The **University Management & Attendance System (UFT)** project aims to revolutionize the management of academic processes within a university by leveraging modern web technologies. Through the development of this system, we have addressed key functionalities that cater to the needs of various stakeholders—students, faculty, and administration. The project has emphasized iterative development, allowing for adaptability to feedback and enhancements. Despite certain limitations encountered during the development, the experiences gained will guide future improvements and feature expansions. Our commitment to ongoing support and updates will ensure that UFT remains relevant and effective in fulfilling its objectives.

9.2 Bibliography

1. https://github.com/python
2. https://youtu.be/6gNpSuE01qE
3. https://xd.adobe.com/ideas/process/ui-design/
4. https://youtu.be/hKB-YGF14SY
5. https://youtu.be/JxzZxdht-XY
6. https://adminlte.io/
7. https://youtube.com/playlist?list=PLu0W_9lII9ah7DDtYtflgwMwpT3mjXY9\